HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY
FACULTY OF COMPUTER SCIENCE

FINAL PRODUCT

USING SOME EXACT AND HEURISTIC METHOD TO SOLVE THE OPTIMIZATION PROBLEM

Course: Fundamentals of Optimization
Class Code: 152625

Group's members:
Nguyễn Hải Đăng - 20235484
Lê Việt Cường - 20235586
Nguyễn Thái Anh Minh - 20235605
Hoàng Tiến Minh - 20235602

Supervisor: Ph.D. Bui Quoc Trung

Table of Contents

1 Overview of our problem: Thesis Defense Jury Assignment **3**

2 Our methods to solve the problem **3**

2.1 Exact Method . . . 3

2.1.1 Brute Force . . . 3

2.1.2 Constraint Programming . . . 7

2.1.3 Integer Programming . . . 9

2.2 Heuristic Method . . . 11

2.2.1 Greedy Algorithm . . . 11

2.2.2 Local search: Hill Climbing . . . 11

2.3 Meta-heuristic Method . . . 13

2.3.1 Iterated Local Search . . . 13

3 Result, Conclusion **14**

4 Work Assignment of our group **15**

1 Overview of our problem: Thesis Defense Jury Assignment

In our problem, we need to divide N theses and M professors into K committees.

- The instructor for each thesis i is called $t(i)$
- The similarity between 2 theses is called $s(i, j)$
- The similarity between thesis and professor is called $g(i, j)$

The constraints of this problem are:

- The number of theses in each committee must be greater than or equal to **a** and less than or equal to **b**.
- The number of professors in each committee must be greater than or equal to **c** and less than or equal to **d**.
- Professors are not allowed to sit on the committee of their own theses.
- The similarity between the theses in the same committee must be greater than or equal to **e**
- The similarity between the theses and the professors in the committee must be greater than or equal to **f**

Our object is to divide professors into committees such that the total similarity between theses and between theses and professors in a committee must be the greatest.

The input for this problem is:

- Line 1: Enter N, M and K ($1 \leq N \leq 1000, 1 \leq M \leq 200, 1 \leq K \leq 100$)
- Line 2: Enter a, b, c, d, e, f
- Line 2+i (i = 1,...,N): Enter line i of matrix s
- Line N+2+i (i = 1,...,N): Enter line i of matrix g
- Last line: Enter $t(1), ..., t(N)$

2 Our methods to solve the problem

2.1 Exact Method

- **Exact method** is able to find an optimal solution to an optimization problem.
- We use 3 exact methods to solve our problem: Brute Force, Integer Programming, Constraint Programming.

2.1.1 Brute Force

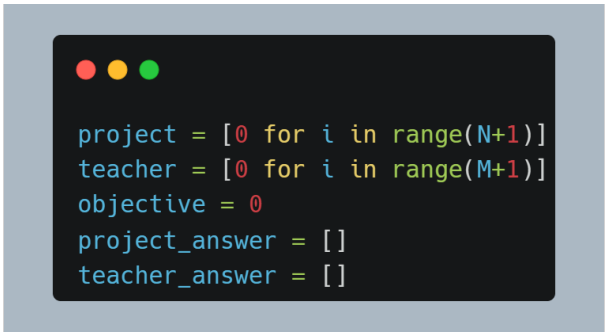
Brute Force is the simplest method in which we enumerate all the combinations of the problem and check which combinations satisfy all the constraints and have the maximal objective function's value.

We will solve this problem by backtracking with variables:

(1) project (project[i] = k, means that project l is presented in room k)

(2) teacher (teacher[j] = k, means that teacher j is presented in room k)

(3) project_answer and teacher_answer: save the optimal answer



```
project = [0 for i in range(N+1)]
teacher = [0 for i in range(M+1)]
objective = 0
project_answer = []
teacher_answer = []
```

Figure 1: Variables

And the input data is read as below:



```
def input_data(filename):
    with open(filename) as text:
        [N, M, K] = [int(x) for x in text.readline().split()]
        [a, b, c, d, e, f] = [int(x) for x in text.readline().split()]
        s = [[0 for x in range(N + 1)]]
        for i in range(N):
            r = [int(x) for x in text.readline().split()]
            s.append([0] + r)
        g = [[0 for x in range(N + 1)]]
        for j in range(1, M + 1):
            r = [int(x) for x in text.readline().split()]
            g.append([0] + r)
        t = [0] + [int(x) for x in text.readline().split()]
    return N, M, K, a, b, c, d, e, f, s, g, t

N, M, K, a, b, c, d, e, f, s, g, t = input_data('data_size_12.txt')
```

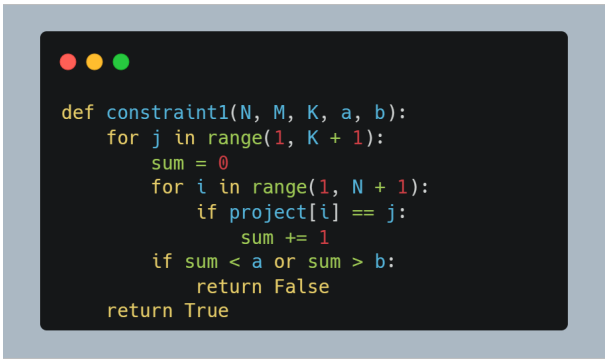
Figure 2: Input

The problem is defined as follows:

There are N projects and M teachers and K rooms. Each project and each teacher must be assigned to exactly one room. There are 5 constraints on this allocation:

(1) The number of projects in each room should be between a and b:

$$a \leq count.(project[i] = k) \leq b,\ \forall i \in \{1, ..., N\},\ \forall k \in \{1, ..., K\}$$



```
def constraint1(N, M, K, a, b):
    for j in range(1, K + 1):
        sum = 0
        for i in range(1, N + 1):
            if project[i] == j:
                sum += 1
        if sum < a or sum > b:
            return False
    return True
```

Figure 3: Constraint 1

(2) The number of teachers in each room should be between c and d:

$$c \leq count.(teacher[k] = l) \leq d,\ \forall j \in \{1, ..., M\},\ \forall k \in \{1, ..., K\}$$

```
def constraint2(N, M, K, c, d):
    for j in range(1, K + 1):
        sum = 0
        for i in range(1, M + 1):
            if teacher[i] == j:
                sum += 1
        if sum < c or sum > d:
            return False
    return True
```

Figure 4: Constraint 2

(3) The project and its assigned teacher should not be in the same room:

$$project[i] \neq teacher[t[i]],\ \forall i \in \{1, ..., N\}$$

```
def constraint3(N, M, K):
    for i in range(1, N + 1):
        if project[i] ==
teacher[t[i]]:
            return False
    return True
```

Figure 5: Constraint 3

(4) The similarity between any two projects in the same room should be more than or equal to e:

$$\forall project[i] = project[j] : s[i][j] \geq e,\ \forall i, j \in \{1, ..., N\}$$

```
def constraint4(N, M, K, e):
    for i in range(1, N + 1):
        for j in range(i + 1, N + 1):
            if project[i] == project[j]:
                if s[i][j] < e:
                    return False
    return True
```

Figure 6: Constraint 4

(5) The similarity between a teacher and a project in the same room should be greater than or equal to f:

$$\forall project[i] = teacher[j] : g[i][j] \geq f,\ \forall i \in \{1, ..., N\},\ \forall j \in \{1, ..., M\}$$

```
def constraint5(N, M, K, f):
    for i in range(1, N + 1):
        for j in range(1, M + 1):
            if project[i] == teacher[j]:
                if g[j][i] < f:
                    return False
    return True
```

Figure 7: Constraint 5

We use the function "calvalue" to calculate the total “similarity between projects” and “similarity between project and teacher"

```
def calvalue(N, M, K):
    value = 0
    for i in range(1, N):
        for k in range(i + 1, N + 1):
            if project[i] == project[k]:
                value += s[i][k]

    for i in range(1, M + 1):
        for j in range(1, N + 1):
            if project[j] == teacher[i]:
                value += g[i][j]

    return value
```

Figure 8: Calculate total value of similarity

We create 2 functions "Try1" and "Try2" to implement the backtracking algorithm:

- **Try1:** generates all possible combinations of assigning projects to each room
- **Try2:** For each project assignment from Try1, it moves to the Try2 function to generate all possible combinations of assigning teachers to each room.

  +) Try2 will verifies whether all constraints are satisfy.

  +) If they are, the algorithm will calculate the value of the current combination and check if it is better than the current best solution.

  +) The best solution is stored in project_answer and teacher_answer lists.

```python
def Try1(k):
    global objective
    global N, M, K, a, b, c, d, e, f, count
    for i in range(1, K + 1):
        project[k] = i
        if k == N:
            Try2(1)
        else:
            Try1(k + 1)

def Try2(k):
    global objective
    global N, M, K, a, b, c, d, e, f, count
    global project_answer, teacher_answer
    for i in range(1, K + 1):
        teacher[k] = i
        if k == M:
            if (constraint1(N, M, K, a, b) and constraint2(N, M, K, c, d) and
                constraint3(N, M, K)       and constraint4(N, M, K, e)    and
                constraint5(N, M, K, f)):
                    p = calvalue(N, M, K)
                    if p > objective:
                        objective = p
                        project_answer = project[:]
                        teacher_answer = teacher[:]
        else:
            Try2(k + 1)
```

Figure 9: Backtracking Algorithm

2.1.2 Constraint Programming

First, we model the problem using **Ortools** library. The model is defined as followed:

(1) Decision variables:

$x[i,j] = 1$, if thesis i is in committee j, else $x[i,j] = 0$, $\forall i \in \{1,...,N\}, \forall j \in \{1,...,K\}$

$y[i,j] = 1$, if professor i is in committee j, else $y[i,j] = 0$, $\forall i \in \{1,...,M\}, \forall j \in \{1,...,K\}$

$x_{\text{int}}[i]$ is the committee for thesis i

$y_{\text{int}}[i]$ is the committee for professor i

$\text{student}[i,j] = 1$, if student i and j will present in the same room, else $\text{student}[i,j] = 0$

$\text{prof}[i,j] = 1$, if professor i and thesis j will present in the same room, else $\text{student}[i,j] = 0$

```python
x_bin = [[model.NewIntVar(0,1, 'x_bin['+str(i)+']['+str(j)+']') for j in range(K)] for i in range(N)]
x_int = [model.NewIntVar(0,K-1, 'x_int['+str(u)+']') for u in range(N)]
y_bin = [[model.NewIntVar(0,1, 'y_bin['+str(m)+']['+str(n)+']') for n in range(K)] for m in range(M)]
y_int = [model.NewIntVar(0,K-1, 'y_int['+str(v)+']') for v in range(M)]

student = [[model.NewIntVar(0,1, 'student['+str(i)+']['+str(j)+']') for j in range(N)] for i in range(N)]
prof = [[model.NewIntVar(0,1, 'prof['+str(m)+']['+str(n)+']') for n in range(N)] for m in range(M)]
```

Figure 10: CP variables

(2) We model our constraints and implement the code for each

$$a \leq \sum_{i=1}^{N} x[i,j] \leq b, \forall j \in \{1,...,K\}$$

$$c \leq \sum_{i=1}^{M} y[i,j] \leq d, \forall j \in \{1,...,K\}$$

```
for j in range(K):
  model.Add(a <= sum([x_bin[i][j] for i in range(N)]))
  model.Add(b >= sum([x_bin[i][j] for i in range(N)]))

for j in range(K):
  model.Add(c <= sum([y_bin[i][j] for i in range(M)]))
  model.Add(d >= sum([y_bin[i][j] for i in range(M)]))
```

Figure 11: Theses / Professors constraints

```
for i in range(N):
  model.Add(x_int[i]!=y_int[t[i]-1])
```

Figure 12: Thesis i and its professor are not in the same committee

Assign value to $x[i,j]$ and $y[i,j]$:

- If $x_{int}[i] = j \rightarrow x[i,j] = 1, \forall i \in \{1,...,N\}, j \in \{1,...,K\}$, else $x[i,j] = 0$
- If $y_{int}[i] = j \rightarrow y[i,j] = 1, \forall i \in \{1,...,M\}, j \in \{1,...,K\}$, else $y[i,j] = 0$

Then, add each constraint for minimum similarity between these and between these and professors:

```
for i in range(N-1):
  for j in range(i+1, N):
    model.Add(x_int[i]==x_int[j]).OnlyEnforceIf(student[i][j])
    model.Add(x_int[i]!=x_int[j]).OnlyEnforceIf(student[i][j].Not())
    model.Add(e<=s[i][j]).OnlyEnforceIf(student[i][j])

for m in range(M):
  for i in range(N):
    model.Add(y_int[m]==x_int[i]).OnlyEnforceIf(prof[m][i])
    model.Add(y_int[m]!=x_int[i]).OnlyEnforceIf(prof[m][i].Not())
    model.Add(f<=g[m][i]).OnlyEnforceIf(prof[m][i])
```

Figure 13: Similarity

(3) Finally, we define our objective function as follow:

```
objective_terms = []
for i in range(N-1):
    for j in range(i+1,N):
        objective_terms.append(s[i][j] * student[i][j])

for i in range(M):
    for j in range(N):
        objective_terms.append(g[i][j] * prof[i][j])
```

Figure 14: Objective function: Maximize the similarity

We obtain the model with all variables, constraints and objective function, which are added to the GLOP solver of Ortools to solve and gain solution.

2.1.3 Integer Programming

With the same approach with Constraint Programming, we will model our problem in a different way:

1. First, we set our variable:

   - $x[i,j] = 1$: thesis i is presented in committee j,
     $\forall i \in \{1,...,N\}, j \in \{1,...,K\}$, else $x[i,j] = 0$
   - $y[i,j] = 1$: professor i is assigned to committee j,
     $\forall i \in \{1,...,M\}, j \in \{1,...,K\}$, else $y[i,j] = 0$
   - $z[i_1,i_2,m] = 1$: thesis i_1 and i_2 are presented at committee m,
     $\forall i_1,i_2 \in \{1,...,N\}, m \in \{1,...,K\}$, else $z[i_1,i_2,m] = 0$
   - $p[i,j,m] = 1$: thesis i and professor$j$ are presented at committee m,
     $\forall i \in \{1,...,N\}, j \in \{1,...,M\}, m \in \{1,...,K\}$, else $p[i,j,m] = 0$

2. We use 10 constraints for this problem using Integer Programming:

   (a) Number of theses in each committee should be in between a and b:

   $$a \leq \sum_{i=1}^{N} x[i,j] \leq b, \forall j \in \{1,...,K\}$$

   (b) Number of professors in each committee should be in between c and d:

   $$c \leq \sum_{i=1}^{M} y[i,j] \leq d, \forall j \in \{1,...,K\}$$

   (c) Each thesis is presented in only 1 committee:

   $$\sum_{j=1}^{K} x[i,j] = 1, \forall i \in \{1,...,N\}$$

   (d) Each professor is assigned in only 1 committee:

   $$\sum_{j=1}^{K} y[i,j] = 1, \forall i \in \{1,...,M\}$$

   (e) The professor and his/her thesis cannot be in the same committee:

   $$x[i,j] + y[t(i),j] \leq 1, \forall i \in \{1,...,N\}, j \in \{1,...,M\}$$

   (f) The similarity between theses in 1 committee must be greater than or equal to **e**:

   $$x[i_1,m] + x[i_2,m] \leq \frac{s(i,j)}{e} + 1, \forall i_1, i_2 \in \{1,...,N\}, m \in \{1,...,K\}$$

   (g) The similarity between theses and professors in 1 committee must be greater than or equal to **f**:

   $$x[i,m] + y[j,m] \leq \frac{g(i,j)}{f} + 1, \forall i \in \{1,...,N\}, j \in \{1,...,M\}, m \in \{1,...,K\}$$

   (h) Set $z[i_1,i_2,m] = 1$ if both theses i_1 and i_2 are presented in the same committee m:

   $$\forall i_1, i_2 \in \{1,...,N\}, m \in \{1,...,K\}$$

   $$\begin{cases} x[i_1,m] + x[i_2,m] \leq 2z[i_1,i_2,m] \\ z[i_1,i_2,m] \geq x[i_1,m] + x[i_2,m] - 1 \\ z[i_1,i_2,m] = z[i_2,i_1,m] \end{cases}$$

   (i) Set $p[i,j,m] = 1$ if both thesis i and professor j are presented in the same committee m:

   $$\forall i \in \{1,...,N\}, j \in \{1,...,M\}, m \in \{1,...,K\}$$

   $$\begin{cases} x[i,m] + y[j,m] \leq 2p[i,j,m] \\ p[i,j,m] \geq x[i,m] + y[j,m] - 1 \end{cases}$$

   (j) Set $p[i,t(i),m] = 0$: thesis i and the professor of i ($t(i)$) are not in the same committee

3. Lastly, we define our objective function as follow:

$$\sum_{m=1}^{K}\left[\sum_{i_1=1}^{N-1}\sum_{i_2=i_1+1}^{N}\Big(z[i_1,i_2,m]\cdot s[i_1,i_2]\Big)+\sum_{i=1}^{N}\sum_{j=1}^{M}\Big(p[i,j,m]\cdot g(j,i)\Big)\right]$$

We add our model to the SCIP solver in Ortools to solve and gain the solution.

2.2 Heuristic Method

Unlike the exact method, this method uses the experience to solve instead of enumerating all the configuration, so we could find a feasible solution at a reasonable computational cost.

2.2.1 Greedy Algorithm

Greedy algorithms can have us reach the optimal value very fast. The following is our greedy strategy:

- **First, we divide theses to K committees by 2 rounds:**
  Round 1:
  Start with an empty committee, add theses until that committee reaches theses → add theses to the next committee until the last committee.

  1. Start with the first committee (m= 1), add theses (say theses 1).
  2. Arrange the remaining theses in the order of total similarity compared to the existing theses descending, add to the current committee the theses have a degree of highest similarity.
  3. Repeat until the committee has "a" theses, then add others theses to the next committee until the last committee.

  Round 2:
  1. Start with the first remaining thesis, sort committees in the order of total similarity compared to that thesis descending and add the thesis to the highest similarity committee. If that committee already has b theses, add this thesis to the 2nd highest and so on until the end of the thesis.
  2. Thus, theses were all divided into committees according to the criterion of achieving the highest total similarity between theses.

- **Secondly, we divide professors to K committees by 2 rounds:**
  Round 1: Starting with committee 1, sort and add the highest similarity to the committee with the condition that the professor doesn't instruct any thesis in the committee. Repeat until the number of professors in the committee reaches c, then move on to the next committee.

  Round 2: Starting with the first remaining professor, repeat the steps of sorting, adding like round 2 of adding theses until the until the last professors.

2.2.2 Local search: Hill Climbing

Local Search starts from an initial solution and evolves that single solution into a mostly better and better solution. It uses a single search path of solutions, not a search tree. At each solution in this path it evaluates a number of moves on the solution and applies the most suitable move to take the step to the next solution. It does that for a high number of iterations until it's terminated.

This is our strategy of Hill Climbing algorithm:

1. First, we generate the initial state of theses and professors:
   The state here is an object that includes 2 attributes. The first one is the list "project". The other is the list named "teacher".
   - project[i]=l, means that project i is presented in room l
   - teacher[j]=l, means that teacher j is presented in room l

```
class State():
    def __init__(self):
        self.thesis = [0 for i in range(0, N + 1)]
        self.professor = [0 for i in range(0, M + 1)]
```

Figure 15: Data structure: State

2. We try to random that 2 lists such that it satisfies 2 constraints
   - The number of projects in each room must be between a and b
   - The number of teachers in each room must be between c and d
3. We have function cal_value to calculate the objective function and function cal_penalty to calculate the number of penalty

```
def calvalue(self, N, M, K):
    sum = 0
    for i in range(1, N + 1):
        for k in range(i + 1, N):
            if self.thesis[i] == self.thesis[k]:
                sum += s[i][k]

    for i in range(0, N):
        for j in range(0, M):
            if self.thesis[i] == self.professor[j]:
                sum += g[i][j]
    return sum

  def cal_penalty(self):
        penalty = 0
        penalty += self.constraint3() + self.constraint4() + self.constraint5()
        return penalty
```

Figure 16: Calculate value and penalty of the current configuration

4. After that we find the neighbors by replace 2 elements of projects lists and find the highest successor

```python
def Highest_Successor(state: State):
    new = state.Copy_state()
    total = -1000000000
    temp_thesis = []
    temp_profes = []

    for i in range(1, N + 1):
        for j in range(i + 1, N + 1):
            new.thesis[i], new.thesis[j] = new.thesis[j], new.thesis[i]
            p = new.calvalue(N, M, K)
            if p > total:
                total = p
                temp_thesis = new.thesis[:]
                temp_profes = new.professor[:]
            new.thesis[i], new.thesis[j] = new.thesis[j], new.thesis[i]

    new.project = temp_thesis
    new.teacher = temp_profes
    return new
```

Figure 17: Highest successor

5. Finally, we use the Hill Climbing algorithm to move to the Highest Successor if the neighbor value is greater than the current value and the penalty of the neighbor is smaller

$$\sum_{i=1}^{N-1}\sum_{j=i+1}^{N}\Big(student[i,j]\cdot s[i,j]\Big)+\sum_{i=1}^{N}\sum_{j=1}^{M}\Big(prof[i,j]\cdot g(j,i)\Big)$$

2.3 Meta-heuristic Method

2.3.1 Iterated Local Search

It is just the updated version of local search where we use a loop to randomize about 100 initial states. And we try hill climbing from each of those states and get the best feasible solution.

```python
def iterated():
    try1 = HillClimbing()
    for i in range(100):
        try2 = HillClimbing()
        if try2.calvalue(N,M,K) > try1.calvalue(N,M,K):
            try1 = try2
    return try1
```

Figure 18: Iterated local search

3 Result, Conclusion

RUN TIME						
	Backtrack	Constraint Programming	Mixed Integer Programming	Greedy	Local Search	Iterated Local search
N = 6	0.003	0.03	0.02	0.004	< 0.001	0.02
N = 10	0.065	0.05	0.03	0.004	0.001	0.12
N = 11	0.125	0.06	0.08	0.009	0.001	0.21
N = 15		0.45	0.36	0.12	0.002	1.18
N = 20		0.83	2.36	0.009	0.007	6.18
N = 25		3.75	35.96	0.14	0.13	14.07
N = 50				0.145	0.15	59.48
N = 100				0.064	2.49	
N = 200				0.17		
N = 300				0.35		
N = 500				1.128		
N = 600				1.79		
N = 800				3.37		
N = 1000				5.53		

Figure 19: Run time of sample test cases

OPTIMAL VALUE						
	Backtrack	CP	IP	Greedy	Local Search	Iterated Local search
N = 6	66	66	66	66	59	62
N = 10	145	145	145	145	126	139
N = 11	405	405	405	402	377	397
N = 15		259	259	253	213	239
N = 20		287	287	279	252	264
N = 25		369	369	335	319	326
N = 50				1341	1284	1309
N = 100				2608	2546	2573
N = 200				4289	4121	
N = 300				5545		
N = 500				9751		
N = 600				14833		
N = 800				19550		
N = 1000				22853		

Figure 20: Optimal value by different methods

From the above running time and optimal values of different methods, we can draw the following conclusions:

1. Exact method and Heuristic methods:
   - Exact methods: always gives the optimal values; with large data size, run times is high and sometimes cannot run.
   - Heuristic methods: release the feasible solutions with lower run times compare. They can reach the optimal value with some small sample data.
2. Among exact methods:
   - Small size (N <= 25), Integer and Constraint Programming return optimal solution with reasonable computation costs (< 1 s).

- Brute Force is the worst methods since it has the highest run time, can only run with small size.

3. Among heuristic methods:
   - Greedy algorithm and Iterated local search give the nearest value to the global optima.
   - With the Iterated local search, the result seems to be more accurate but longer run time than the others and still don't satisfy all the constraints.
   - Greedy method run time is more reasonable when the data size grows larger.

4 Work Assignment of our group

WORK ASSIGNMENT

Brute force code	Tiến Minh
LP, CP code	Anh Minh, Tiến Minh
Greedy code	Cường
Local search, Iterated local search	Đăng
Presentation slide	Anh Minh
Testing	Tiến Minh, Đăng, Cường
Report	Anh Minh

Figure 21: Work assignment